Learning to Select Context per Code Diff: An Accuracy–Cost Trade-off for LLM Code Review

Michael Liu (msl196) **Ruiding Feng (rf711)** **Li-Hua Peng (lp880)**
Machine Learning II, Rutgers University

Abstract

We study whether selecting context for each code change can reduce the input cost of LLM-assisted code review without reducing accuracy. Our planned method trains a small context selector while keeping the reviewer fixed. So far, we have prepared a repository-based split of 46,518 ContextCRBench examples, completed a Qwen3B QLoRA training pilot, and implemented fixed, random, and similarity-based context policies. This report describes the audited data and training setup. Evaluation outputs are under review, so we do not yet draw conclusions about adaptation or context selection.

Repository: `https://github.com/michaelsliu3/CS536_Project`

1 Research question

A code reviewer may need an issue description to understand a change's purpose or nearby code to see how it will be used. Giving the model every available field uses more tokens, even when some fields add little useful information. We ask: *Can a learned selector choose context for each code change while maintaining review accuracy and using fewer tokens than fixed context choices?*

ContextCRBench provides code changes together with issue, pull request (PR), and surrounding-code context [1]. Repoformer studies selective retrieval for code completion [2]; we ask whether a separate selector can help with code-review judgments. Following instructor feedback, we focus on the accuracy–cost trade-off.

Our target is accuracy within one percentage point of the best fixed strategy with at least 30% fewer extra context tokens, or higher accuracy at a matched budget. The Qwen3B adaptation pilot checks the data and evaluation pipeline and asks whether fine-tuning improves merge-status prediction. It is separate from the main experiment, which will keep Qwen frozen and train the selector.

2 Data and experimental setup

Task and split. The reviewer returns a JSON answer with the Boolean field `is_merged`. Merge status is our benchmark target, but only a proxy for code quality: a merged change can contain a bug, and a rejection can have nontechnical reasons. We exclude explicit outcome fields and human review answers from prompts. Later edits and quoted solutions in issue and PR text still need a chronology audit.

The downloaded release contains 68,938 JSON records. Cleaning removes 22,375 exact normalized-diff duplicates and excludes 45 records from 16 groups with conflicting labels, leaving 46,518 examples from 99 repositories. With seed 42, we target an 80/10/10 split by repository; all changes from one PR remain together. The saved manifest has disjoint repository sets (Table 1). Unequal repository sizes explain the different example proportions. We use validation for development and reserve test repositories for the final comparison. This split does not rule out near-duplicates or temporal leakage.

Table 1: Retained data in the saved split. True and False denote merge status.

Split	Examples	Repositories	True	False	True (%)
Train	38,097	79	30,281	7,816	79.5
Validation	3,797	10	3,264	533	86.0
Test	4,624	10	3,336	1,288	72.1

The splits differ in both class balance and language mix. Go (23.7%) and Python (17.4%) are the largest training languages, while C# (39.4%) and Rust (27.8%) dominate validation. The fixed 200-example generation sample contains 175 True and 25 False labels from all ten validation repositories. These imbalances make per-class scores and the training-majority baseline necessary alongside accuracy.

A Python example. Training record `final_json/63859.json` is from `pandas-dev/pandas`, PR #53773. Its issue describes a `TypeError` when nested JSON contains `null`. Four verbatim diff lines are shown below; other lines are omitted, and + marks an addition.

```
+         if data is None:
+             return
          if isinstance(data, dict):
              data = [data]
```

The guard returns before processing `None`. Its label, `False`, records merge status rather than whether the fix is correct. This is an input illustration, not a model prediction; the label stays out of the prompt.

Model and adaptation method. The audited pilot uses Qwen2.5-Coder-3B-Instruct, a decoder-only Transformer with 36 layers, hidden size 2,048, and feed-forward size 11,008. Grouped-query attention uses 16 query heads and two key/value heads. The model generates the JSON verdict autoregressively. Although its configuration supports 32,768 positions, our experiment limits each sequence to 2,048 tokens.

QLoRA [3] freezes the base weights in 4-bit NF4 with double quantization and bfloat16 computation. We train low-rank adapters on the attention query, key, value, and output projections and the feed-forward gate, up, and down projections. Rank 16, scaling $\alpha = 32$, and dropout 0.05 give 29,933,568 trainable parameters. The saved run uses paged 8-bit AdamW, learning rate 2×10^{-4}, cosine decay, 3% warmup, and weight decay 0.01. Batch size one with eight accumulation steps gives an effective batch size of eight. Training ran for three epochs (14,289 optimizer steps) on one 32-GB RTX 5090; the final adapter is `checkpoint-14289`.

Inputs contain repository and language metadata, the diff, issue and PR text, and code before and after the change. Training supervises only the JSON answer and end-of-sequence token; prompt tokens are masked from the loss. Right truncation keeps the sequence within 2,048 tokens. After preprocessing, 63.3% of training and 57.2% of validation sequences reach this cap. These are post-truncation lengths, not measurements of how many tokens were discarded.

Evaluation and context policies. Evaluation compares the untouched base, the trained adapter, and a constant prediction chosen from training-label counts on the same saved 200 examples. The two model conditions share prompts, quantization, and deterministic decoding, with at most 2,016 input tokens and 32 generated tokens. Invalid JSON counts as an error and is reported separately. We will compare accuracy, macro-F1, per-class scores, and paired improvements and regressions.

For the frozen-reviewer study, diff-only, all-context, fixed-mask, random, and BM25 similarity policies are implemented. They select among issue text, PR text, and code before and after the change; the diff is always present. Assembled prompts are checked against token budgets. We plan to train a compact Transformer selector from reviewer outcomes on training repositories. The selector has not been trained, and context-policy comparisons remain pending. Fixed masks and budgets will be chosen on validation data.

3 Progress and results still needed

Saved cloud artifacts confirm the training setup and data counts above. Prediction files also exist, but generation and answer parsing are still being checked. We therefore leave Table 2 unfilled. Dashes indicate unfinalized measurements, not zero scores. Training loss alone cannot establish an improvement in generated judgments.

Table 2: Comparison to be completed on the same saved validation sample. The final test partition is reserved.

Method	Accuracy	Macro-F1	Invalid JSON rate
Training-majority constant	—	—	N/A
Untouched Qwen3B	—	—	—
Qwen3B + trained adapter	—	—	—

[To complete: Verify generated answers and parsing, then fill Table 2 and add a training/validation loss plot before submission.]

4 What we learned

Cloud runs made recovery a practical requirement. Checkpoints now retain adapter, optimizer, scheduler, and random states. We changed validation from every 100 to every 1,000 optimizer steps and corrected the restored interval on resume; timing measurements are still needed to assess the effect. The data audit also shows why accuracy alone is insufficient: labels are imbalanced, language mix changes across repositories, and many inputs reach the token cap. An all-context prompt does not guarantee that the reviewer sees every field. We need to check context timing and truncation before interpreting differences between models.

5 Next steps

Our immediate task is to resolve the evaluation checks, fill Table 2, and inspect improvements and regressions on held-out repositories. Pilot findings will be reported separately from the context-selection experiment.

We will then run a small frozen-reviewer comparison to check whether context choices change enough judgments to make per-example selection useful. If they do, we will generate training outcomes and train the selector. Comparisons will include accuracy, extra context tokens, and runtime including selection, followed by repeated seeds and context-type ablations. If the selector does not beat the best fixed policy, we will analyze where context helps or hurts and report that result.

Acknowledgments. We thank Professor Lesia Semenova for feedback on the research question and evaluation. We used OpenAI Codex for coding, debugging, and editing assistance. Our team is responsible for checking the implementation, running the experiments, and interpreting the results.

References

[1] R. Hu et al. Benchmarking LLMs for fine-grained code review with enriched context in practice. *arXiv:2511.07017*, 2025. `https://arxiv.org/abs/2511.07017`.

[2] D. Wu, W. U. Ahmad, D. Zhang, M. K. Ramanathan, and X. Ma. Repoformer: Selective retrieval for repository-level code completion. *ICML*, PMLR 235:53270–53290, 2024.

[3] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *arXiv:2305.14314*, 2023.